



Segurança da
Informação

TRÍADE CIA





Tríade CIA

INTRODUÇÃO

E ARQUITETURA

Neste trabalho, nosso objetivo geral foi ir além da teoria e descrever a concepção e os fundamentos operacionais de um ecossistema real.

Nós desenvolvemos em Python um software que simula um barramento corporativo responsável pelo tráfego de dados sensíveis.



Segurança da Informação





Tríade CIA

INTRODUÇÃO E ARQUITETURA

Para isso, utilizamos uma abordagem de

Security by
Design

Defense in
Depth,

garantindo que as proteções atuem de forma nativa e integrada ao código, mitigando vulnerabilidades desde a base do sistema.



Segurança da Informação





Tríade CIA

Mecanismo Utilizado: Fernet (Cifragem Simétrica AES-128-CBC + Autenticação HMAC-SHA256)

Objetivo: Proteger informações em repouso contra acesso não autorizado por usuários sem privilégios.

Técnica: AES-128 em modo CBC com vetor de inicialização aleatório. HMAC-SHA256 assegura integridade e autenticidade.

PILAR I: CONFIDENCIALIDADE





Tríade CIA

CONFIDENCIALIDADE

```
25     def _carregar_ou_gerar_chave(self):
26         if Path(self.arquivo_chave).exists():
27             with open(self.arquivo_chave, "rb") as f:
28                 chave = f.read()
29             print(f" Chave carregada de: {self.arquivo_chave}")
30         else:
31             chave = Fernet.generate_key()
32             with open(self.arquivo_chave, "wb") as f:
33                 f.write(chave)
34             print(f" Nova chave gerada e salva em: {self.arquivo_chave}")
35
36         self.cipher = Fernet(chave)
37
38     def criptografar(self, dados: str) -> bytes:
39         dados_bytes = dados.encode("utf-8")
40         dados_cifrados = self.cipher.encrypt(dados_bytes)
41         return dados_cifrados
42
```





Tríade CIA

Mecanismo Utilizado: Funções Hash SHA-256 e HMAC-SHA256 Concorrentes para proteção de dados.

Objetivo: Detectar modificações involuntárias ou adulterações maliciosas em trânsito de informações.

Técnica: SHA-256 transforma dados em hash de 256 bits. HMAC adiciona autenticação e não repúdio.

PILAR II: INTEGRIDADE





Tríade CIA

INTEGRIDADE

```
37
38     def verificar_integridade(self, dados: str, hash_esperado: str, metodo: str = "sha256") -> bool:
39         if metodo == "sha256":
40             hash_atual = self.calcular_sha256(dados)
41         elif metodo == "hmac":
42             hash_atual = self.calcular_hmac(dados)
43         else:
44             raise ValueError(f"Metodo desconhecido: {metodo}")
45
46         return hmac.compare_digest(hash_atual, hash_esperado)
47
```





Tríade CIA

Rate Limiting (Janela Deslizante):
Controla requisições simultâneas para
evitar sobrecarga e exaustão de
recursos do sistema.

Backoff Exponencial: Reduz a
sobrecarga em falhas consecutivas,
aumentando intervalos entre tentativas
de reconexão.

Backup com Timestamp: Restaura
dados em caso de falhas críticas,
garantindo continuidade operacional e
resiliência.

PILAR III: DISPONIBILIDADE





Tríade CIA

DISPONIBILIDADE

```
19 class RateLimiter:
20     def __init__(self, max_requisicoes: int = 5, janela_segundos: int = 10):
21         self.max_requisicoes = max_requisicoes
22         self.janela_segundos = janela_segundos
23         self.historico: dict = defaultdict(list)
24         self._lock = threading.Lock()
25
```

```
110
111     @retry_automatico(max_tentativas=3, espera_base=0.5)
112
```





Tríade CIA

MATRIZ RESUMO

01

Confidencialidade

Mecanismo: Fernet (AES-128-CBC). Ativo: Dados sensíveis em leitura/escrita. Cifra dados com chave simétrica e autenticação HMAC.

02

Integridade

Mecanismo: SHA-256 / HMAC-SHA256. Ativo: Payload de mensagens. Garante autenticidade e não repúdio dos dados.

03

Disponibilidade

Mecanismo: Rate Limiting / Backoff / Backup. Ativo: Acesso contínuo e infraestrutura. Limita abusos e restaura serviços.





Tríade CIA

BOAS PRÁTICAS



Revisão de Código

Auditorias periódicas do código-fonte para identificar vulnerabilidades e garantir conformidade com padrões de segurança estabelecidos.



Monitoramento Contínuo

Acompanhamento em tempo real de logs e configuração de alertas para identificar atividades suspeitas e responder rapidamente.



Testes Automatizados

Implementação de testes automatizados de integridade para validar a consistência dos dados e detectar alterações não autorizadas.



Versionamento Seguro

Adoção de políticas de versionamento seguro para controlar alterações e manter histórico completo de modificações no sistema.





Tríade CIA

Dificuldades Operacionais

Adaptação de conceitos criptográficos a cenários lógicos reais. Integração de bibliotecas Python e tratamento de exceções.

Aprendizados Consolidados

Criptografia isolada não assegura legitimidade sem HMAC. Validações físicas e temporais são essenciais. Disponibilidade depende de redundância.



CONCLUSÃO

LIÇÕES APRENDIDAS





Tríade CIA

MUITO OBRIGADA!